



Codexion

Domina la carrera por los recursos antes de que la
fecha límite te domine a ti

*Resumen: compite contra el tiempo en este emocionante desafío de concurrencia.
Orquesta a múltiples personas que programan compitiendo por un número limitado de
dongles USB mediante hilos POSIX, mutexes y planificación inteligente—domina la
sincronización de recursos antes de que llegue el agotamiento.*

Versión: 1.0

Índice general

I.	Introducción	2
II.	Instrucciones sobre la IA	4
III.	Instrucciones generales	7
IV.	Descripción general	8
V.	Reglas globales	9
VI.	Parte obligatoria	12
VII.	Requisitos del Readme	14
VIII.	Entrega y peer-evaluation	16

Capítulo I

Introducción

La programación informática (a menudo llamada simplemente "programación") es el proceso de diseñar y construir software ejecutable para cumplir con tareas informáticas específicas. Implica análisis, generación de algoritmos, evaluación del rendimiento y consumo de recursos de los algoritmos, e implementación de dichos algoritmos en el lenguaje de programación elegido. Los primeros ordenadores se programaban directamente en código máquina, secuencias de instrucciones binarias comprendidas por el hardware. Con el tiempo, se desarrollaron lenguajes de más alto nivel, permitiendo a quien programa expresar ideas de forma más abstracta.

La cultura de la programación va más allá de lo puramente técnico. Mezcla resolución de problemas, creatividad y colaboración en entornos compartidos que van desde comunidades de software libre hasta espacios de co-working y hackathones. En estos lugares, quienes programan no solo escriben código, sino que también depuran, refactorizan y comparten herramientas. Algunos recursos en estos entornos abundan (por ejemplo, el café), mientras que otros — como dongles de hardware especializado o entornos de desarrollo con licencia — son limitados y deben compartirse con cuidado para evitar cuellos de botella.

En la filosofía antigua se debatía sobre la verdad y la existencia, quienes programan hoy debaten sobre paradigmas de programación, patrones de diseño y prácticas éticas del software. ¿Es mejor optimizar para velocidad o para facilidad de mantenimiento? ¿Conviene refactorizar ahora o después de que el producto demuestre su valor? ¿Cómo puede un equipo asegurar un acceso justo a los recursos de desarrollo compartidos sin frenar el progreso?

Históricamente, la programación solía ser una actividad solitaria, pero la evolución de herramientas colaborativas — sistemas de control de versiones, editores en tiempo real y gestores de incidencias distribuidos — ha convertido la programación en un proceso cada vez más social y cooperativo. A la par, la concurrencia y la sincronización se han vuelto habilidades vitales: saber cómo permitir que muchas personas o procesos trabajen en desarrollo sin interferir entre sí es tan importante como escribir el propio código.

En la era moderna, con equipos repartidos por zonas horarias y proyectos que usan hardware escaso o limitado en el tiempo, el desafío no es solo escribir código correcto, sino también diseñar protocolos justos y eficientes para compartir recursos. Esta simulación

modela un escenario así: personas programando en un espacio compartido que requieren dos dongles USB raros para compilar su código cuántico. Deben coordinarse, evitar interbloqueos y prevenir el agotamiento — porque en el mundo de la programación colaborativa, como en la vida, el acceso y el tiempo lo son todo.

Capítulo II

Instrucciones sobre la IA

● Contexto

Durante tu proceso de aprendizaje, la IA puede ayudarte con muchas tareas diferentes. Tómate el tiempo necesario para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyarte con tu trabajo. Sin embargo, siempre debes abordarlas con precaución y evaluar de forma crítica los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca podrás saber con total certeza si tu pregunta está bien formulada o si el contenido generado es el adecuado. Las personas que te rodean son tu recurso más valioso para ayudarte a evitar errores y puntos ciegos.

● Mensaje principal:

- ✚ Utiliza la IA para reducir las tareas repetitivas o tediosas.
- ✚ Desarrolla habilidades de prompting, ya sea para programación o para otros temas, que beneficiarán tu futura carrera.
- ✚ Aprende cómo funcionan los sistemas de IA para anticipar de forma eficiente y evitar los riesgos comunes, sesgos y problemas éticos.
- ✚ Sigue trabajando con tus compañeros para desarrollar tanto habilidades técnicas como habilidades transversales.
- ✚ Utiliza únicamente contenido generado por IA que entiendas completamente y del cual puedas responsabilizarte.

● Reglas para estudiantes:

- Debes tomarte el tiempo necesario para explorar las herramientas de IA y comprender cómo funcionan, para poder utilizarlas de manera ética y reducir los sesgos potenciales.
- Debes reflexionar sobre tu problema antes de dar instrucciones a la IA. Esto te ayuda a escribir preguntas, instrucciones o conjuntos de datos más claros, detallados y relevantes utilizando un vocabulario preciso.

- Debes desarrollar el hábito de revisar, cuestionar y probar sistemáticamente cualquier contenido generado por la IA.
- Debes buscar siempre la revisión de otras personas, no te limites a confiar en tu propia validación.

● Resultados de esta etapa:

- Desarrollar habilidades de prompting tanto generales como de ámbito específico.
- Aumentar tu productividad con un uso eficaz de las herramientas de IA.
- Seguir fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

● Comentarios y ejemplos:

- Ten en cuenta que la IA puede no tener la respuesta correcta porque esa respuesta no esté ni siquiera en Internet. Además, si te da soluciones incorrectas, intenta no insistir y busca ayuda entre las personas que te rodean. Vas a ahorrarte tiempo y vas a sumar en comprensión.
- Vas a enfretarte con frecuencia a situaciones (como exámenes o evaluaciones) donde debes demostrar una comprensión real. Prepárate, sigue construyendo tanto tus habilidades técnicas como transversales.
- Explicar tu razonamiento y debatir con otras personas suele revelar lagunas en tu comprensión de un concepto. Prioriza el aprendizaje entre pares.
- Lo normal es que la herramienta de IA que utilices no conozca tu contexto específico (a menos que se lo indiques), así que te dará respuestas genéricas. Si buscas información más adecuada y más precisa en relación a tu entorno cercano, confía en el resto de estudiantes.
- Donde la IA tiende a generar la respuesta más probable, el resto de estudiantes puede proporcionar perspectivas alternativas y matices valiosos. Confía en la comunidad de 42 como un punto de control de calidad.

✓ Buenas prácticas:

Le pregunto a la IA: "¿Cómo pruebo una función de ordenación?" Me da algunas ideas. Las pruebo y reviso los resultados con otra persona. Refinamos el enfoque de manera conjunta.

✗ Mala práctica:

Le pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad. Suspenso mi proyecto.

✓ **Buenas prácticas:**

Utilizo la IA para ayudarme a diseñar un parser. Luego, reviso la lógica con otra persona. Encontramos dos errores y lo reescribimos juntos: mejor, más limpio y comprendiendo al 100

✗ **Mala práctica:**

Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no puedo justificarlo y suspendo mi proyecto.

Capítulo III

Instrucciones generales

- El proyecto deberá estar escrito en C.
- El proyecto debe estar escrito siguiendo la Norma. Si tienes archivos o funciones adicionales, estas deberán estar incluidas en la verificación de la Norma y tendrás un 0 si hay algún error de norma en cualquiera de ellos.
- Las funciones no deben terminar de forma inesperada (segfault, bus error, double free, etc), excepto en el caso de comportamientos indefinidos. Si esto sucede, el proyecto será considerado no funcional y recibirás un 0 durante la evaluación.
- Toda la memoria asignada en la pila (heap) deberá liberarse adecuadamente cuando sea necesario. No se permitirán leaks de memoria.
- Si el enunciado lo requiere, se deberá entregar un `Makefile` que compilará tus archivos fuente a la salida requerida con las flags `-Wall`, `-Werror` y `-Wextra`. También se deberá utilizar `cc` y, por supuesto, el `Makefile` no debe hacer `relink`.
- El `Makefile` entregado debe contener al menos las normas `$(NAME)`, `all`, `clean`, `fclean` y `re`.
- Para entregar los bonus del proyecto se deberá incluir una regla `bonus` en el `Makefile`, en la que se añadirán todos los headers, librerías o funciones que estén prohibidas en la parte principal del proyecto. Los bonus deben estar en archivos distintos `× _bonus.{c/h}`. La parte obligatoria y los bonus se evalúan por separado.
- Si el proyecto permite el uso de la `libft`, se deberá copiar su fuente y sus `Makefile` asociados en un directorio `libft` con su correspondiente `Makefile`. El `Makefile` del proyecto debe compilar primero la librería utilizando su `Makefile`, y después compilar el proyecto.
- Es recomendable crear programas de prueba para el proyecto, aunque este trabajo **no será entregado ni evaluado**. Esto ofrece la oportunidad de verificar que el programa funciona correctamente durante las evaluaciones. Y sí, está permitido utilizar estas pruebas durante cualquier evaluación.
- Entrega el trabajo en el repositorio `Git` asignado. Solo el trabajo de tu repositorio `Git` será evaluado. Si el proyecto tiene que ser evaluado por Deepthought, la evaluación se realizará después de las evaluaciones personales. Si, durante la evaluación de Deepthought, se encuentra un error se, se interrumpirá su evaluación.

Capítulo IV

Descripción general

Esto es lo que hay que saber si quieres se desea completar con éxito este proyecto:

- Una o más **personas que programan** se sientan en un hub circular inclusivo de co-working.
En el centro, hay un **Compilador Cuántico** compartido.
- Las personas alternan entre **compilar, depurar o refactorizar**.
Mientras compilan, no están depurando ni refactorizando;
mientras depuran, no están compilando ni refactorizando;
y, por supuesto, mientras refactorizan, no están compilando ni depurando.
- Hay **dongles USB** en la mesa. Hay **tantos dongles como personas que programan**.
- Compilar código cuántico requiere dos dongles conectados simultáneamente, uno en cada mano: cada persona toma su dongle izquierdo y derecho para compilar.
- Cuando una persona termina de compilar, devuelve ambos dongles a la mesa y comienza a depurar.
Una vez terminada la depuración, comienza a refactorizar. La simulación se detiene cuando alguien **se agota** debido a la falta de compilación.
- Todas las personas necesitan compilar regularmente y nunca deberían agotarse.
- Las personas no se comunican entre sí.
- Las personas no saben si otra está a punto de agotarse.
- Por supuesto, ¡deben evitar el agotamiento!

Capítulo V

Reglas globales

Se debe escribir **un solo programa** que cumpla las siguientes reglas:

- ¡Las variables globales están prohibidas!
- El programa debe recibir los siguientes argumentos (todos obligatorios):
`number_of_coders time_to_burnout time_to_compile time_to_debug`
`time_to_refactor number_of_compiles_required dongle_cooldown scheduler`
 - `number_of_coders`: el número de personas que programan y también el número de dongles.
 - `time_to_burnout` (en milisegundos): si una persona no comienza a compilar dentro de `time_to_burnout` milisegundos desde el inicio de su última compilación o desde el comienzo de la simulación, se agota.
 - `time_to_compile` (en milisegundos): el tiempo que tarda alguien en compilar. Durante ese tiempo, debe sostener dos dongles.
 - `time_to_debug` (en milisegundos): el tiempo que se dedicará a depurar.
 - `time_to_refactor` (en milisegundos): el tiempo empleado en refactorizar. Tras completar la fase de refactorización, la persona intentará de inmediato adquirir dongles y comenzar a compilar de nuevo.
 - `number_of_compiles_required`: si todas las personas han compilado al menos este número de veces, la simulación se detiene. De lo contrario, se detiene cuando alguien se agota.
 - `dongle_cooldown` (en milisegundos): después de ser liberado, un dongle está **no disponible** hasta que su periodo de enfriamiento haya pasado.
 - `scheduler`: la política de arbitraje usada por los dongles para decidir quién los obtiene cuando varias personas los solicitan.
El valor debe ser exactamente uno de: `fifo` o `edf`.
`fifo` significa *First In, First Out*: el dongle se concede a la persona cuya solicitud llegó primero.
`edf` significa *Earliest Deadline First* con `deadline = last_compile_start + time_to_burnout`.

- Cada persona tiene un número que va de 1 a `number_of_coders`.
- La persona con número 1 se sienta junto a la número `number_of_coders`.
Cualquier número N se sienta entre $N - 1$ y $N + 1$.



Recordatorio: todos los argumentos son obligatorios. Rechaza entradas no válidas como números negativos, valores no enteros o un scheduler diferente de `fifo` o `edf`.

Sobre los logs del programa:

- Cualquier cambio de estado de una persona debe formatearse del siguiente modo:
 - `timestamp_in_ms X has taken a dongle`
 - `timestamp_in_ms X is compiling`
 - `timestamp_in_ms X is debugging`
 - `timestamp_in_ms X is refactoring`
 - `timestamp_in_ms X burned out`

Se debe sustituir `timestamp_in_ms` por la marca temporal actual en milisegundos y `X` por el número de la persona que programa.

- Un mensaje de estado mostrado no debe mezclarse con otro mensaje.
- Un mensaje que anuncie que alguien se agotó no debe mostrarse más de 10 ms después del momento real del agotamiento.
- Una vez más, ¡las personas deben evitar agotarse!

Ejemplo del formato esperado del log:

```
0 1 has taken a dongle
1 1 has taken a dongle
1 1 is compiling
201 1 is debugging
401 1 is refactoring
402 2 has taken a dongle
403 2 has taken a dongle
403 2 is compiling
603 2 is debugging
803 2 is refactoring
1204 3 burned out
```



Requisito de precisión: los logs de agotamiento deben mostrarse dentro de los 10 ms posteriores al momento real del agotamiento. Se debe permitir una tolerancia mínima durante las pruebas, ya que el hardware y la planificación del sistema operativo pueden afectar ligeramente las mediciones de tiempo.



Consideración sobre temporización: para reducir el impacto del hardware en las mediciones de rendimiento, se recomienda usar tiempo de CPU en lugar del reloj de tiempo real cuando sea posible. No obstante, para este proyecto, las mediciones en tiempo real utilizando `gettimeofday()` son aceptables y recomendadas por simplicidad.

Capítulo VI

Parte obligatoria

Nombre de programa	codexion
Archivos a entregar	Makefile, *.c, *.h en el directorio coders/
Makefile	NAME, all, clean, fclean, re
Argumentos	number_of_coders time_to_burnout time_to_compile time_to_debug time_to_refactor number_of_compiles_required dongle_cooldown scheduler
Funciones autorizadas	pthread_create, pthread_join, pthread_mutex_init, pthread_mutex_lock, pthread_mutex_unlock, pthread_mutex_destroy, pthread_cond_init, pthread_cond_wait, pthread_cond_timedwait, pthread_cond_broadcast, pthread_cond_destroy, gettimeofday, usleep, write, malloc, free, printf, fprintf, strcmp, strlen, atoi, memset
Se permite usar libft	No
Descripción	Personas que programan con hilos y mutexes (C)

Las reglas específicas de la parte obligatoria son:

- Cada persona que programa debe representarse mediante un hilo (utilizando `pthread_create`).
- Hay un dongle entre cada par de personas. Por lo tanto, si hay varias personas, cada una tendrá un dongle a su izquierda y otro a su derecha. Si solo hay una persona, solo debe haber un dongle sobre la mesa.
- Para evitar que se dupliquen dongles, debes proteger el estado de cada dongle con un mutex (`pthread_mutex_t`). Puede utilizarse una variable de condición (`pthread_cond_t`) para gestionar colas de espera.
- **El enfriamiento del dongle es obligatorio:** después de que una persona libere un dongle, este no puede volver a tomarse hasta que hayan transcurrido los

milisegundos indicados por `dongle_cooldown`.

- **El arbitraje justo es obligatorio:** cuando varias personas soliciten el mismo dongle, este debe concederse según el `scheduler`.
Con `fifo`, se atiende las solicitudes en orden de llegada.
Con `edf`, se atiende a la persona con el deadline de agotamiento más próximo (es decir, `last_compile_start + time_to_burnout`).
- El programa debe garantizar que todo el mundo esté bien: nadie debe agotarse bajo el planificador `edf`, siempre que los parámetros sean viables.
- Debe existir un hilo **monitor** separado que detecte el agotamiento con precisión y detenga la simulación. El log de agotamiento debe imprimirse dentro de los 10 ms posteriores al momento real en que se produce.
- El registro debe estar serializado, de modo que nunca se mezclen dos mensajes en una misma línea (usa un mutex para proteger la salida).
- La simulación se detiene cuando alguien se agota o cuando todas las personas han compilado al menos `number_of_compiles_required` veces.
- Tu código debe compilar con `-Wall -Wextra -Werror -pthread`.
- Se debe implementar una cola de prioridad (heap) para la planificación FIFO/EDF (C89 no tiene una biblioteca estándar para esto).
- Toda memoria debe asignarse y liberarse correctamente (sin fugas de memoria).

Ejemplo de ejecución de la simulación:

```
0 1 has taken a dongle
2 1 has taken a dongle
2 1 is compiling
202 1 is debugging
402 1 is refactoring
405 2 has taken a dongle
406 2 has taken a dongle
406 2 is compiling
606 2 is debugging
806 2 is refactoring
900 3 has taken a dongle
902 3 has taken a dongle
902 3 is compiling
1102 3 is debugging
1302 3 is refactoring
1505 4 burned out
```



Este ejemplo ilustra la secuencia de acciones para múltiples personas. Aquí se observa cómo cada acción de `compiling` está precedida por dos líneas de "has taken a dongle", y cómo el mensaje "burned out" aparece en el momento en el que alguien no cumple su deadline.

Capítulo VII

Requisitos del Readme

Debe incluirse un archivo `README.md` en la raíz del repositorio Git. Su propósito es permitir que cualquier persona que no esté familiarizada con el proyecto (pares, personal, responsables de selección, etc.) pueda entender rápidamente de qué trata el proyecto, cómo ejecutarlo y dónde encontrar más información sobre el tema.

El `README.md` debe incluir, como mínimo:

- La primera línea debe estar en cursiva y decir: *Este proyecto ha sido creado como parte del currículum de 42 por <login1>[, <login2>[, <login3>[...]]]*.
 - Una sección de "**Descripción**" que presente claramente el proyecto, incluyendo su objetivo y una breve visión general.
 - Una sección de "**Instrucciones**" que contenga cualquier información relevante sobre compilación, instalación y/o ejecución.
 - Una sección de "**Recursos**" que enumere referencias clásicas relacionadas con el tema (documentación, artículos, tutoriales, etc.), así como una descripción del uso de IA, especificando para qué tareas y en qué partes del proyecto se ha utilizado.
- Podrían requerirse secciones adicionales dependiendo del proyecto (por ejemplo, ejemplos de uso, lista de características, decisiones técnicas, etc.).

Cualquier contenido extra requerida se listará explícitamente a continuación.

Para este proyecto, el archivo `README.md` también debe incluir:

- Una sección "**Blocking cases handled**" que describa todos los problemas de concurrencia gestionados en la solución creada (por ejemplo, prevención de interbloqueos y condiciones de Coffman, prevención de inanición, gestión del cooldown, detección precisa del agotamiento y serialización del log).
- Una sección "**Thread synchronization mechanisms**" explicando las primitivas de hilos específicas usadas en la implementación (`pthread_mutex_t`, `pthread_cond_t`, implementación personalizada de eventos) y cómo coordinan el acceso a recursos compartidos (dongles, logging, estado del monitor). Debe incluir ejemplos de cómo se evitan las condiciones de carrera y cómo se logra una comunicación thread-safe entre las personas que programan y el monitor.



La elección del idioma queda a discreción personal. Se recomienda escribir en inglés, pero no es obligatorio.

Capítulo VIII

Entrega y peer-evaluation

Entrega tu proyecto en tu repositorio `Git` como de costumbre. Solo se evaluará durante la defensa el contenido que esté dentro de tu repositorio. No dudes en comprobar dos veces los nombres de tus archivos para asegurarte de que son correctos.

Recode instructions

Durante la evaluación, es posible que se solicite una ligera **modificación del proyecto**. Esto puede consistir en ajustar ligeramente el comportamiento, modificar unas cuantas líneas de código o incorporar una característica fácil de implementar.

Puede que este paso **no sea necesario en todos los proyectos**, pero hay que tenerlo en cuenta si así se especifica en la hoja de evaluación.

Este paso sirve para verificar la comprensión real de una parte específica del proyecto. La modificación se puede realizar en cualquier entorno de desarrollo que se elija (por ejemplo, la configuración habitual), y debería ser factible en unos pocos minutos, a menos que se defina un plazo específico como parte de la evaluación.

Por ejemplo, se puede pedir hacer una pequeña actualización en una función o *script*, modificar lo que se vería en pantalla o ajustar una estructura de datos para almacenar nueva información, etc.

Los detalles (alcance, objetivo, etc.) se especificarán cada **hoja de evaluación** y pueden variar de una evaluación a otra para el mismo proyecto.